OSSP-PRACTICAL-WEEK7

TASK1: Write a program that prints the addresses of code, global, static, heap, and stack variables. Analyze the Linux process address space layout and explain each memory segment.

Source code:

```
  GNU nano 7.2
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>

int global_var = 10;

void show_addresses()
{
    int stack_var = 20;
    static int static_var = 30;
    int *heap_var = malloc(sizeof(int));

    *heap_var = 40;

    printf("Process Address Space\n");
    printf("--------------------\n");

    printf("Code   address : %p\n", (void *)show_addresses);
    printf("Global address : %p\n", (void *)&global_var);
    printf("Static address : %p\n", (void *)&static_var);
    printf("Heap   address : %p\n", (void *)heap_var);
    printf("Stack  address : %p\n", (void *)&stack_var);

    printf("\nProcess ID: %d\n", getpid());

    printf("\nPress Enter to exit...");
    getchar();

    free(heap_var);
}

int main()
{
    show_addresses();

    return 0;
}
```

Output:

```
sathwika@DESKTOP-AA8K5MG:~/OS-Debugging-Project$ gcc -g -Wall -Wextra -o debug_demo src/debug_demo.c
sathwika@DESKTOP-AA8K5MG:~/OS-Debugging-Project$ ./debug_demo
=== OSSP Debugging Demonstration ===

[1] Testing Segmentation Fault (Null Pointer)

Segmentation fault!
Program terminated at src/debug_demo.c:9

gdb backtrace:
#0  main () at src/debug_demo.c:28

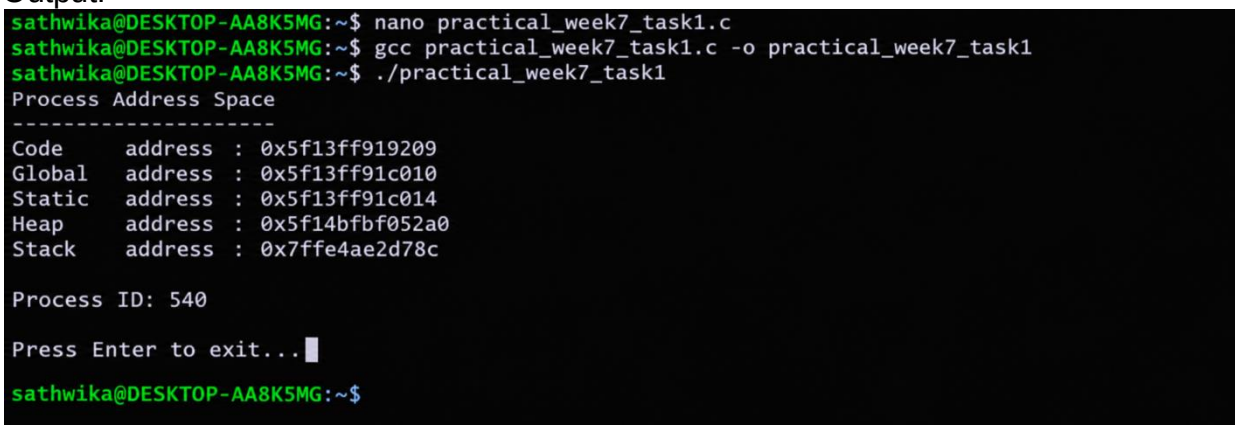
[2] Testing Memory Bug
Writing beyond allocated memory...

sathwika@DESKTOP-AA8K5MG:~/OS-Debugging-Project$
```

Observation: I learned how a Linux process is organized into different memory segments such as code, global, static, heap, and stack. I learned how to print the memory addresses of variables belonging to these segments and understood that dynamically allocated memory is stored in the heap while local variables are stored in the stack.

Task2: Make use of the /proc/[pid]/maps file and memory analysis tools to study the memory organization of a running process. Prepare a report explaining virtual memory mappings.

Output:



Observation: I learned how to examine the virtual memory layout of a running Linux process using /proc/[PID]/maps. I understood how memory mappings represent code, data, heap, stack, and shared libraries, along with their read, write, and execute permissions. I also learned how tools such as pmap can be used to analyze process memory usage.